

Predicting Book Ratings from Catalogue Metadata

An end-to-end machine learning pipeline and web application

Sashank Nadavaluru · Applied MSc in Data Science & Artificial Intelligence · DSTI — School of Engineering · Instructor: Hanna Abi Akl

1. Introduction

If you know everything about a book except what is written inside it, how well can you guess how much readers will like it? This is not a purely academic question: a publisher setting a print run, or a retailer allocating shelf space, has to make that call before a single review comes in. All they have is the catalogue record — page count, imprint, language, publication date, the author line.

The dataset contains 11,128 rows and 12 attributes from a Goodreads-style catalogue: two identifiers (bookID and both ISBNs), descriptive fields (title, authors, publisher, language_code, publication_date, num_pages), two engagement measures (ratings_count, text_reviews_count), and the target, average_rating. The data was supplied raw, which mattered a great deal — much of the work went into establishing what the numbers actually mean before any model saw them. Framed formally this is supervised regression: predict a continuous rating between roughly 1 and 5. The pipeline covers exploratory analysis, cleaning, feature engineering, a comparison of four regressors, and deployment behind a Streamlit interface.

I state the headline result at the outset, because interpreting it is the most interesting part of the project: the best model explains around 15% of the variance in book ratings. That is modest, and I argue in Section 3.3 that it is close to the ceiling this data can support — and that reporting it honestly is more useful than engineering it upward.

2. Methodology

2.1 Exploratory Data Analysis

The file would not load cleanly: unescaped commas inside titles are read as extra columns, so the parser fails on those rows. Loading with `on_bad_lines='skip'` discards them rather than aborting, leaving ~11,123 usable rows from 11,127 — an acceptable price for a DataFrame that loads reliably.

The inspection pass produced the finding that shaped everything afterwards. `isnull().sum()` returned zero across all twelve columns, which on a raw file is suspicious rather than reassuring. `describe()` explained why: `average_rating` had a minimum of 0.0, and `num_pages` could also be 0. Missing data was present, but encoded as placeholder zeros rather than NaN, so no null check would ever find it. Crucially the two zeros do not mean the same thing. A genuine zero-star average would require every reader to rate at the very bottom, which effectively never happens once a book has ratings at all — a zero there means *unrated*. A zero in `num_pages` means the count was never entered; the book obviously has pages. Two further problems: the page-count header is " num_pages" with leading whitespace, so `df['num_pages']` raises a `KeyError`; and `publication_date` was still a string, concealing impossible dates such as 11/31/2000 and 6/31/1982.

On distributions: `average_rating` has a mean of 3.93 and a standard deviation of only 0.35, with the middle half of books between 3.77 and 4.14 — characteristic of self-selected rating data, since readers choose books they expect to enjoy. Because the target has so little spread, predicting 3.93 for every book is already a strong baseline, a fact that reappears in the results. The engagement columns are shaped very differently: `ratings_count` has a median of 745 against a mean of 17,943 and a maximum of 4.6 million, and page counts show the same skew (median 299, maximum 6,576). That skew is why median rather than mean imputation is used below.

2.2 Data Cleaning

Issue	Rule applied	Rows affected
Whitespace in column headers	<code>str.strip()</code> on all names	—
Impossible calendar dates	Parse with <code>errors='coerce'</code> , drop NaT	2
<code>average_rating == 0</code>	Drop the row	~25
<code>num_pages == 0</code>	Median impute, add <code>pages_missing_flag</code>	~76

Table 1: Cleaning rules, applied in a fixed order, and the rows each affected.

Zero ratings are dropped while zero page counts are imputed, because treating both alike would be a mistake in either direction. A missing rating is a missing *target*: the row cannot supervise training, and imputing it would mean inventing the exact quantity the model is meant to learn. A missing page count is a missing *feature*: that row still carries a real observed rating alongside a valid publisher, author and review count, and discarding it would throw away a genuine label

over one absent field. The general principle is to drop missing targets and impute missing features when the rest of the record remains informative.

The median was used for imputation because page counts have a long right tail — omnibus editions run to several thousand pages — which pulls the mean upward. A binary `pages_missing_flag` accompanies the imputed value; without it an imputed row is indistinguishable from a book that genuinely has the median page count, and the model loses any chance to learn whether missingness itself carries signal. (It turned out to be the least important feature — a conclusion from evidence, not an assumption.) None of this is outlier removal: removing a genuine 2.1 or 4.9 rating would discard valid observations and narrow the very distribution being modelled. These rows were removed because their values are not measurements at all.

2.3 Feature Engineering and Selection

Several promising columns are strings, so the engineering step extracts numeric signal from them, producing a nine-column feature matrix.

Feature	Origin	Rationale
<code>num_pages</code>	Raw (cleaned)	Proxy for format and genre; novellas, novels and reference works differ systematically
<code>ratings_count</code>	Raw	Popularity and how established the title is
<code>text_reviews_count</code>	Raw	Depth of engagement, distinct from raw volume
<code>author_count</code>	Engineered	Contributors counted by splitting authors on /
<code>title_length</code>	Engineered	Character count; a cheap proxy for title style
<code>publication_year</code>	Engineered	Era effects, from the parsed date
<code>language_code_encoded</code>	Engineered	Label-encoded language
<code>publisher_encoded</code>	Engineered	Label-encoded imprint
<code>pages_missing_flag</code>	Engineered	Marks median-imputed page counts

Table 2: The nine-column feature matrix, with the rationale for each column.

Author count over author identity. The `authors` field joins writers and illustrators with a slash (J.K. Rowling/Mary GrandPré). Encoding author identity would almost certainly be more predictive, but with thousands of names — most appearing once or twice — the encoding would be sparse and would risk leaking target information if computed naively across the full dataset. Counting contributors is weaker but structurally safe, separating solo works from collaborations and illustrated editions.

Label encoding over one-hot encoding. This is the choice I would defend most carefully, since it is only valid given the models selected. Label encoding implies an ordering that does not exist — nothing makes publisher 1,500 “greater than” publisher 20 — which genuinely damages a linear model fitting one coefficient across an alphabetical artefact. Tree models split on thresholds and can isolate arbitrary ranges of codes, so numeric proximity is never read as similarity. With ~2,200 distinct publishers, one-hot would have expanded the matrix to thousands of sparse columns for ~11,000 rows, inviting overfitting; label encoding keeps it compact, and the two models expected to win are both tree ensembles. The honest caveat is that this partly handicaps the Linear Regression baseline. Finally, the identifiers `bookID`, `isbn` and `isbn13` were excluded deliberately: being unique per row they are useless for generalisation and actively harmful, since a deep tree could memorise individual books through them.

2.4 Model Selection, Comparison and Evaluation

The data was split 80/20 with `random_state=42`, giving 8,876 books for fitting and 2,220 for evaluation. The split is well balanced (target mean 3.94 train against 3.95 test; standard deviations 0.298 and 0.292). Fixing the seed matters more than it appears: the two leading models are separated by 0.0002 in MAE, and without it each run would draw a different partition and the comparison would measure split noise rather than model quality. I did not stratify — the target is continuous, there are no classes to balance, and given how tightly ratings cluster a plain random split already yields closely matched distributions.

Four regressors were compared, spanning a range of complexity:

- **Linear Regression** — the interpretable baseline; assumes a linear, additive relationship, so it cannot capture interactions such as page count mattering only for certain publishers.
- **Decision Tree** — captures non-linearity and interactions natively, but a single unpruned tree readily memorises the training set.
- **Random Forest** — many trees on bootstrap samples with random feature subsets, averaged; since individual trees overfit in uncorrelated ways, averaging reduces variance.

- **Gradient Boosting** — trees fitted sequentially on the residuals of the ensemble so far; typically strong on tabular data, at the cost of more sensitive tuning.

All four were fitted with scikit-learn defaults on the identical split and scored with three metrics. MAE gives the average error in rating points and is the most directly interpretable; RMSE penalises large errors more heavily, so the gap between them indicates occasional bad misses; R^2 measures the fraction of variance explained, where 0 equals always predicting the mean.

3. Results

3.1 Model comparison

Model	MAE	RMSE	R^2
Gradient Boosting	0.1983	0.2689	0.1504
Random Forest	0.1981	0.2705	0.1398
Linear Regression	0.2122	0.2831	0.0580
Decision Tree	0.2819	0.3790	-0.6878

Table 3: Performance on the held-out test set (2,220 books), 80/20 split, random_state=42.

The ordering matches theory, but two entries deserve comment. The Decision Tree's negative R^2 is the most striking number: it performed *worse* than always predicting the training mean — better to ignore every feature and return 3.94. Left unpruned, it split until its leaves were nearly pure, fitting noise specific to those 8,876 books, and its MAE is 42% worse than the winner's. Including it demonstrates precisely why the ensembles exist.

Gradient Boosting versus Random Forest is genuinely close, and Random Forest actually wins on MAE (0.1981 against 0.1983) — a gap of 0.0002 of a rating point, which is not a meaningful difference. But Gradient Boosting leads on RMSE (0.2689 against 0.2705) and R^2 (0.1504 against 0.1398). When two models tie on MAE but one leads on RMSE, the difference lies in the tails, since RMSE squares errors before averaging: both are equally accurate on the typical book, but Gradient Boosting makes fewer badly wrong predictions. For an application returning a single number to a user, avoiding the occasional large error beats a fourth-decimal improvement in the average, so Gradient Boosting was selected.

Linear Regression's weak R^2 of 0.058 carries the caveat raised earlier. Part of the disadvantage is real — the relationships are not linear and additive — but part is self-inflicted, since it was handed label-encoded publisher and language columns whose arbitrary ordering it cannot interpret. A fairer baseline would use one-hot or target encoding; I would expect improvement, though not enough to overtake the ensembles.

3.2 What the model actually learned

Feature	Importance
num_pages	0.2524
ratings_count	0.1878
title_length	0.1776
publisher_encoded	0.1615
publication_year	0.0644
author_count	0.0582
text_reviews_count	0.0549
language_code_encoded	0.0364
pages_missing_flag	0.0068

Table 4: Gradient Boosting feature importances, summing to 1.0.

Page count dominates at 0.25, which surprised me until I stopped reading it as “longer books are better.” It is really a proxy for format: books of 80, 300 and 1,200 pages tend to be children's books, novels and reference or omnibus editions respectively, and those categories genuinely attract different rating behaviour. ratings_count at 0.19 needs the most care, and is where a causal reading would go badly wrong — the model has *not* discovered that more ratings raise a score. Three non-causal explanations fit better: confounding (good books attract more readers, so quality drives both), selection (readers pick books they expect to like, so widely read books draw a favourable audience), and statistical stabilisation (a book with eight ratings can average 4.9 or 2.1 by chance and so populates the extremes, whereas one with 80,000 is pinned near the population mean). The model may largely be learning that last effect — that a low count predicts high *variance* rather than high quality.

title_length at 0.18 is higher than expected for what is on its face a trivial feature; my reading is that it captures series and edition conventions rather than prose, since long titles belong to series instalments with subtitles and volume markers, or to academic works with descriptive subtitles. The pattern across the whole table is consistent: the model

relies on how big, how visible, whose imprint, and what the title looks like. Every one of those remains metadata. None of them is quality.

3.3 Interpreting an R^2 of 0.15

The model explains about 15% of the variance and leaves roughly 85% unexplained — the correct result rather than a disappointing one. What drives a rating is the reading experience: the prose, the plot, the characters, the ending, whether it arrived at the right cultural moment. None of that is present anywhere in this dataset — no text, no genre label, no blurb, not even a subject tag. The model is asked to predict subjective quality from the shipping label rather than the contents of the box. Seen that way, 15% is a reasonable haul: format, imprint and visibility carry some real signal. But the ceiling for this feature set is low, and no amount of tuning will move it much, because the limitation is in the data rather than the algorithm.

It is worth noting what a much higher number would have implied. An R^2 of 0.9 from these nine features should draw suspicion rather than congratulation, since that pattern usually indicates target leakage. In practical terms, an MAE of 0.198 means predictions land within about 0.2 rating points of the truth on average — respectable, but the honest comparison is against always predicting the mean, which is already fairly accurate given how tightly ratings cluster. The model's real contribution is the 15% of variation it captures beyond that: useful for ranking or triage, not for confidently scoring an individual title.

4. Deployment

The selected model is served through a Streamlit application (`app.py`) that accepts book metadata and returns a predicted rating. The central engineering challenge is that the model does not understand books — it understands a nine-column numeric row assembled in a specific order by specific encoders, while a user reasonably wants to type “Scholastic Inc.” rather than the integer 1,847. The obvious approach of saving the model and re-deriving preprocessing in the app is a trap: if the app recomputes the imputation median from a different snapshot, or assembles columns in a different order, nothing fails, no exception is raised, and the app returns a plausible number between 1 and 5 that is quietly wrong. To prevent this, the notebook serialises a bundle rather than a bare model, saved to `models/book_rating_bundle.joblib`:

```
artifact = {
    "model": trained[best_name],          # Gradient Boosting
    "language_encoder": language_encoder, # fitted LabelEncoder
    "publisher_encoder": publisher_encoder, # fitted LabelEncoder
    "feature_cols": FEATURE_COLS,         # exact column order
    "median_pages": median_pages_value,    # 302.0, training median
    "model_name": best_name,
    "fallback_language": "eng",
    "fallback_publisher": "Vintage",
}
```

The app derives all preprocessing from this bundle, and enforces column ordering by indexing the constructed row with `bundle["feature_cols"]`, so a mismatch raises an error rather than silently transposing columns. A `LabelEncoder` also holds a fixed set of training categories, so an unseen publisher raises `ValueError: y contains previously unseen labels` — routine rather than exceptional given free-text input. The app maps unknown categories to a frequent training category (`eng`, `Vintage`) and displays a warning. The warning is the essential half: substituting silently would reintroduce the failure mode above, where the user cannot distinguish an approximated prediction from a genuine one.

The form collects the raw fields a person would actually know: title, authors, language code, page count, ratings count, text reviews count, publisher and publication date, with feature engineering happening inside the app immediately before prediction. Three conveniences were added: an OpenLibrary lookup that searches by title or ISBN and auto-fills page count, publisher, author line and publication year; an expandable panel showing the exact engineered feature row the model received, making the prediction inspectable rather than opaque; and sensible defaults pre-filled in every field so the app demonstrates itself on first load.

One limitation should be stated plainly: OpenLibrary does not expose rating volumes, so when the lookup is used, `ratings_count` and `text_reviews_count` are filled with placeholders. Since `ratings_count` is the second most important feature, a prediction from an auto-filled record demonstrates the interface rather than estimating that specific book; entering real engagement figures manually gives a genuine prediction.

[Insert screenshot of the running Streamlit app here — form filled in, with the predicted rating displayed]

Figure 1: The Streamlit interface returning a prediction for a sample book.

```
git clone https://github.com/NadavaluruSashank/Book-Rating-Prediction.git
cd Book-Rating-Prediction && python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt && streamlit run app.py
```

A pre-trained bundle is committed to the repository, so the app runs without re-training. `requirements.txt` pins `scikit-learn==1.9.0`, the version that produced the bundle, since loading a pickled estimator under a different version produces compatibility warnings and is not guaranteed to behave identically.

5. Conclusion

This project built a complete pipeline from a raw, unclean CSV through to a deployed web application. Gradient Boosting was selected over three alternatives, achieving an MAE of 0.198 and an R^2 of 0.150 on held-out data, served through a Streamlit interface that reuses the exact training-time preprocessing.

The most valuable thing I took from it was not the model but the finding that the reasoning behind the cleaning decisions mattered more than the choice of algorithm. Recognising that zeros in this file encode two entirely different kinds of missingness — an absent label in one column, an unrecorded attribute in another — changed what the dataset was; treating both alike, by dropping every zero or imputing every zero, would have trained the model on a subtly corrupted target that no metric would have revealed. The second lesson concerns reporting a weak result. There was a real temptation to keep tuning until R^2 looked more impressive, but the constraint here is informational, not algorithmic: metadata simply does not contain most of what determines whether people enjoy a book, and recognising a ceiling imposed by the data is the more useful skill than chasing it with more complex models.

Directions for improvement

- **Author-level features.** The largest omission. An author's mean rating across their other works would almost certainly be the strongest available predictor, since reception is consistent within a catalogue — but it must be computed within the training fold only, or it leaks test-set targets and produces an inflated, meaningless score.
- **Text features from titles.** `title_length` ranks third despite discarding every actual word. TF-IDF over title tokens would capture genre signals — “Chronicles,” “Handbook,” “Illustrated” carry real information.
- **Better categorical encoding.** Frequency or smoothed target encoding would preserve more signal than arbitrary integer codes without the dimensionality cost of one-hot, and would make the linear baseline a fair comparison.
- **Hyperparameter tuning.** Every model used library defaults. A cross-validated search over `n_estimators`, `learning_rate` and `max_depth`, plus XGBoost or LightGBM, would yield incremental gains — single-digit percentages rather than a transformation.
- **Weighting by rating volume.** A book with 12 ratings and one with 200,000 are treated identically despite the first being far noisier. Weighting by `ratings_count`, or excluding books below a threshold, would train on more reliable targets.
- **Prediction intervals.** Given the genuine uncertainty, returning “3.8 to 4.2” would be more honest than a point estimate given to two decimals.

The broader conclusion is that the path to a materially better model runs through richer data — genre tags, descriptions, author histories, cover art — rather than more sophisticated algorithms applied to the same twelve columns.

Repository and References

GitHub: <https://github.com/NadavaluruSashank/Book-Rating-Prediction> — contains the full Jupyter notebook, the Streamlit source, the trained model bundle, the dataset, a README and a pinned `requirements.txt`.

- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830. <https://scikit-learn.org/>
- The pandas development team. *pandas: powerful Python data analysis toolkit*. <https://pandas.pydata.org/>
- Streamlit Inc. *Streamlit documentation*. <https://streamlit.io/>
- Friedman, J. H. (2001). Greedy Function Approximation: A Gradient Boosting Machine. *Annals of Statistics*, 29(5), 1189–1232.
- Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.
- OpenLibrary Search API, used for the metadata auto-fill feature. <https://openlibrary.org/developers/api>